Análisis sintáctico y evaluación de atributos

Santiago Villamizar · Juan Sebastián Ramírez
Proyecto final · Enunciado del 9 de octubre de 2026

Introducción

La implementación recibe gramáticas libres de contexto por terminal y determina si admiten análisis LL(1) y SLR(1). El segundo modo analiza una secuencia con signo y calcula una suma ponderada mediante una definición dirigida por la sintaxis (SDD). Los algoritmos se ejecutan en Python 3 con la biblioteca estándar.

Representación, FIRST y FOLLOW

Una producción contiene una cabeza y una tupla de símbolos. La tupla vacía representa ε. El símbolo inicial siempre es S. Se conserva el orden de entrada y se usan conjuntos para los terminales, los no terminales y los valores de FIRST y FOLLOW.

La sección 4.4.2 de Aho et al. establece dos cálculos de punto fijo. FIRST inicia cada terminal con su propio símbolo. Para cada cuerpo se recorre el prefijo anulable, se incorporan los terminales posibles y se añade ε si todo el cuerpo es anulable. FOLLOW inicia con $ en FOLLOW(S). Para A → αBβ se incorporan los terminales de FIRST(β) a FOLLOW(B). Si β es anulable, también se propaga FOLLOW(A). Cada cálculo termina después de una pasada sin cambios.

Tabla predictiva y análisis LL(1)

Según la sección 4.4.3, M[A, a] recibe A → α para cada terminal a de FIRST(α). Si α es anulable, también se registra la producción en las columnas de FOLLOW(A). El diccionario de celdas conserva las producciones originales. La comprobación por pares detecta intersecciones FIRST/FIRST, FIRST/FOLLOW y dos alternativas anulables, incluso cuando FOLLOW está vacío.

La sección 4.4.4 usa una pila inicial [$, S]. Para un no terminal se consulta la tabla y se apila el cuerpo en orden inverso. Para un terminal se exige coincidencia con la entrada. El reconocimiento termina tras consumir ambos marcadores. Una celda vacía o un terminal distinto causa rechazo.

Interfaz de terminal

El modo principal calcula ambas clasificaciones antes de seleccionar un analizador. Si ambos aplican, T y B abren lotes sucesivos y Q termina. Una línea vacía cierra el lote y la línea e representa ε. La opción --sets imprime los conjuntos antes de la clasificación.

Construcción y ejecución SLR(1)

La sección 4.5 presenta reducciones, desplazamientos y conflictos. En la sección 4.6.2 se aumenta la gramática con S′ → S. Cada ítem guarda una producción y la posición del punto. CLOSURE agrega ítems iniciales de cada no terminal después del punto. GOTO desplaza el punto sobre el símbolo indicado y aplica CLOSURE. Los conjuntos inmutables identifican los estados de la colección canónica.

La sección 4.6.4 coloca desplazamientos por transición terminal y reducciones por ítem completo en las columnas de FOLLOW de su cabeza. S′ → S· agrega aceptación sobre $. GOTO almacena transiciones no terminales. ACTION conserva conjuntos de acciones para identificar conflictos sin aplicar precedencias. Según la sección 4.6.3, un desplazamiento consume entrada y apila un estado. Una reducción retira tantos estados como símbolos tiene el cuerpo y apila GOTO para la cabeza.

Verificación de S → AA, A → aA | b

FOLLOW(S) = {$} y FOLLOW(A) = {a, b, $}. La primera A puede ir seguida de a o b y la segunda hereda $. Las reducciones r1, r2 y r3 usan S → AA, A → aA y A → b, respectivamente. La prueba compara la colección, las transiciones, la tabla y cada paso con los valores de esta verificación.

Estado e ítems LR(0)	a	b	$	A	S
0 S′ → ·S, S → ·AA, A → ·aA, A → ·b	d3	d4		1	2
1 S → A·A, A → ·aA, A → ·b	d3	d4		5	
2 S′ → S·			acepta		
3 A → a·A, A → ·aA, A → ·b	d3	d4		6	
4 A → b·	r3	r3	r3		
5 S → AA·			r1		
6 A → aA·	r2	r2	r2		

Traza para abb

Pila antes de la acción	Entrada	Acción
0	abb$	Desplazar a 3
0 3	bb$	Desplazar a 4
0 3 4	b$	Reducir A → b. GOTO(3, A) = 6
0 3 6	b$	Reducir A → aA. GOTO(0, A) = 1
0 1	b$	Desplazar a 4
0 1 4	$	Reducir A → b. GOTO(1, A) = 5
0 1 5	$	Reducir S → AA. GOTO(0, S) = 2
0 2	$	Aceptar

SDD de secuencias con signo

La gramática tiene tres no terminales y seis producciones. L es recursivo. Cada elemento x pesa 1 y cada y pesa 2. La SDD suma los pesos multiplicados por su posición, con el signo del prefijo.

```
S → +L | −L
L → DL | ε
D → x | y
```

Conjuntos y tabla calculados por el programa

Símbolo	FIRST	FOLLOW	+	-	x	y	$
S	{+, -}	{$}	S → +L	S → -L			
L	{x, y, ε}	{$}			L → DL	L → DL	L → ε
D	{x, y}	{x, y, $}			D → x	D → y	

Cada terminal tiene su propio conjunto FIRST. Las celdas en blanco son errores. Se puede reproducir la tabla con build_ll1_table y los conjuntos con --sets.

Atributos

Símbolo	Atributo	Tipo	Clase	Significado
S	total	entero	sintetizado	Resultado de la secuencia completa
L	step	entero	heredado	Posición con signo del elemento actual
L	total	entero	sintetizado	Suma ponderada del sufijo
D	weight	entero	sintetizado	Peso del elemento
+, -, x, y	lexval	carácter	sintetizado	Carácter de la entrada

Reglas semánticas

Producción	Reglas semánticas
S → +L	L.step = 1, S.total = L.total
S → -L	L.step = -1, S.total = L.total
$L_0 \rightarrow D\ L_1$	L_1.step = L_0.step + signo(L_0.step) L_0.total = L_0.step × D.weight + L_1.total
L → ε	L.total = 0
D → x	D.weight = peso(x.lexval) = 1
D → y	D.weight = peso(y.lexval) = 2

En $L_0 \rightarrow D\ L_1$, L_0 es la cabeza y L_1 el hijo. El signo de step es 1 si step es positivo y -1 si es negativo. Las reglas heredadas usan una constante o un atributo heredado de la cabeza. La SDD es L-atribuida y el atributo step impide que sea S-atribuida (secciones 5.2.3 y 5.2.4).

Evaluación y verificación

Evaluador genérico

El árbol se reconstruye de forma iterativa a partir de las producciones que emitió el analizador LL(1) (sección 5.1.2). Cada regla contiene destino, dependencias y función. Se traducen las posiciones locales a identificadores de nodo y se crea la lista de adyacencia (sección 5.2.1). Una cola de vértices con grado de entrada cero produce el orden topológico (sección 5.2.2). Si quedan vértices pendientes, hay un ciclo y se detiene la evaluación antes de ejecutar reglas.

Resultados de las pruebas

Entrada SDD	+	-	-yx	+xyx	+xz	+ seguido de 2000 x
Resultado	0	0	-4	aceptada	no	2 001 000

Las cadenas + y - son las más cortas. Para comprobar la circularidad se cambió temporalmente L.step = 1 por L.step = L.total en S → +L. Con +x, L.total también depende de L.step. La ejecución produjo «SDD is circular.» y detuvo el lote. La regla original se conserva en el programa.

La suite ejecutó 32 pruebas con 32 éxitos y 0 fallos. Se comparó literalmente la salida de los tres ejemplos oficiales, el caso solo LL(1), la selección T/B/Q y --sets. También se comprobaron la colección LR(0), ambos tipos de conflicto SLR, las aristas del grafo y los atributos del árbol de +xyx.

Manual Development of the SDD

Given the original grammar and its semantic rules, manually develop the following for the input string +xyx:

1. Annotated parse tree.
2. Attribute dependency graph.
3. A valid topological evaluation order.
4. Step-by-step attribute evaluation and final result.

Annotated parse tree

Attribute dependency graph

Topological evaluation order and attribute calculations

Problemas, conclusiones y referencias

Problemas y decisiones

Situación	Solución aplicada
Dos alternativas anulables con FOLLOW vacío	La comprobación LL(1) compara las alternativas aunque ninguna se inserte en una celda común.
La misma producción llega a una celda por FIRST y FOLLOW	Se unen los símbolos de anticipación antes de insertar la producción. Así se registra una sola vez en cada celda.
Acciones incompatibles en una celda SLR	ACTION guarda el conjunto completo de acciones. Un conjunto de tamaño mayor que uno impide seleccionar ese analizador.
Reducción por ε	El ítem se crea completo. La reducción deja intacta la pila antes de consultar GOTO para la cabeza.
Secuencias profundas	Los analizadores, la reconstrucción del árbol y la evaluación usan estructuras iterativas. La prueba de SDD procesa 2000 elementos.
Regla circular en la SDD	El ordenamiento detecta los vértices que no llegan a grado de entrada cero. Se informa el ciclo antes de calcular valores.

Conclusiones

Los conjuntos FIRST y FOLLOW conectan la gramática con las decisiones de ambos analizadores. LL(1) decide por el no terminal pendiente y el símbolo de entrada. SLR(1) combina el estado de la colección LR(0) con el símbolo de entrada y limita las reducciones mediante FOLLOW. La comparación de conflictos permite elegir un analizador con las reglas de la gramática original.

La evaluación de atributos se realiza sobre un árbol ya reconocido. Las reglas de la SDD determinan qué valores calcular y el grafo determina cuándo están disponibles.

Referencias

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2.ª ed.). Addison-Wesley. ISBN 978-0-321-48681-3. Secciones 4.4.2–4.4.4, 4.5, 4.6.2–4.6.4, 5.1.1–5.1.2 y 5.2.1–5.2.4.

Castro, A. A., Guerra, C. A., Rodríguez, O. A., y Rico, S. R. (2026, 9 de octubre). Project. Enunciado de SI2002, ST0270 y CM0081.